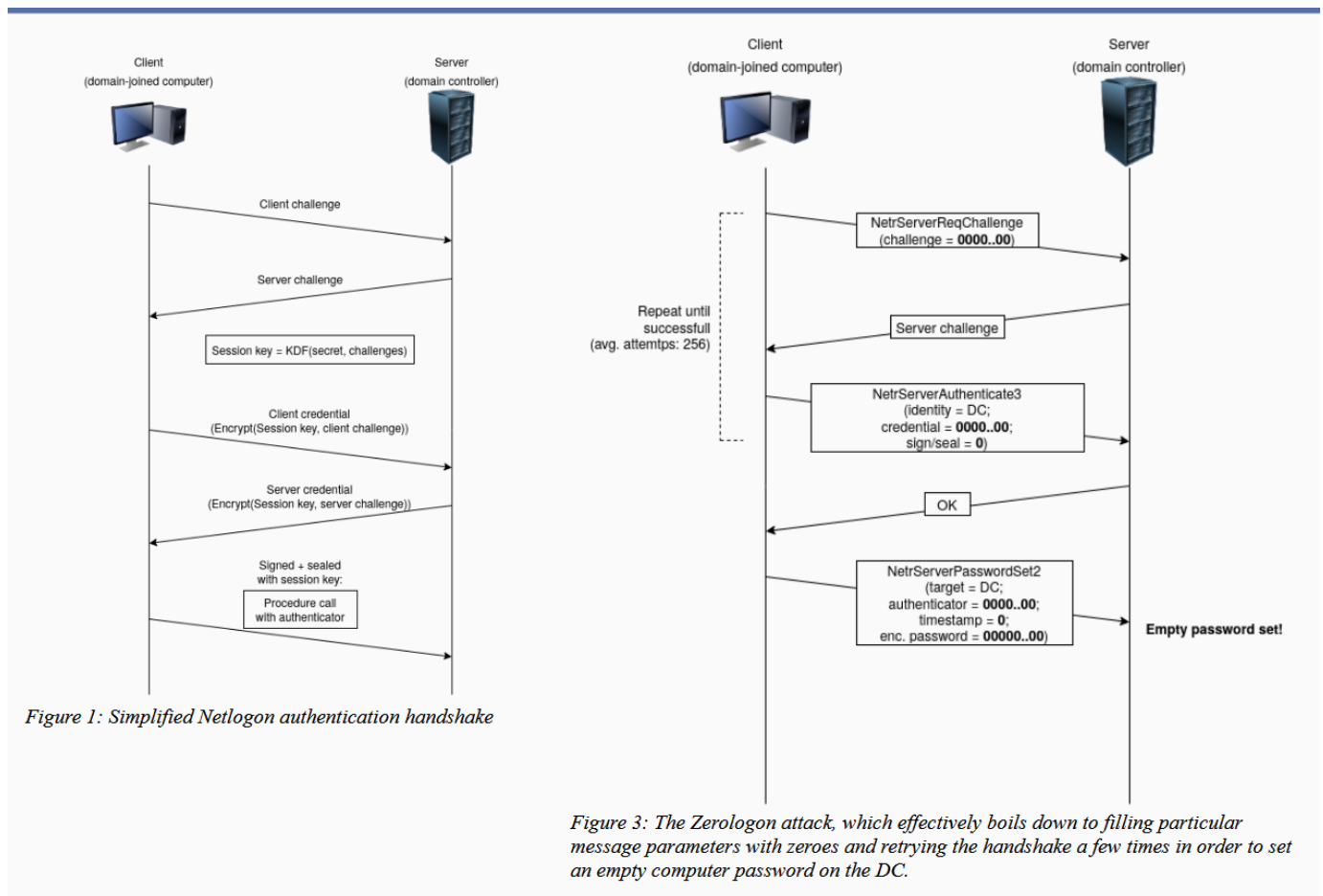


TryHackMe - ZeroLogon

Zero Logon is a purely statistics based attack that abuses a feature within MS-NRPC (Microsoft NetLogon Remote Protocol), MS-NRPC is a critical authentication component of Active Directory that handles authentication for User and Machine accounts. In short -- the attack mainly focuses on a poor implementation of Cryptography. To be more specific, Microsoft chose to use AES-CFB8 for a function called ComputeNetlogonCredential, which is normally fine, except they had hard coded the Initialization Vector to use all zeros instead of a random string. When an attacker sends a message only containing zeros with the IV of zero, there is a 1-in-256 chance that the Ciphertext will be Zero.



Step 1. The client creates a NetrServerReqChallenge and sends it off [Figure 1. Step 1]. This contains the following values:

1. The DC
2. The Target Device (Also the DC, in our case)
3. A Nonce (In our case is 16 Bytes of Zero).

Step 2. The server receives the NetrServerReqChallenge, the server will then generate it's own Nonce (This is called the Server Challenge), the server will send the Server Challenge back. [Figure 1. Step 2]

Step 3. The client (us) will compute it's NetLogon Credentials with the Server Challenge provided [Figure 1. Step 3]. It uses the NetrServerAuthenticate3 method which requires the following parameters:

1. A Custom Binding Handle (Impacket handles this for us, it's negotiated prior)
2. An Account Name (The Domain Controller's machine account name. ex: DC01\$)
3. A Secure Channel Type (Impacket sort of handles this for us, but we still need to specify it: [nrpc.NETLOGON_SECURE_CHANNEL_TYPE.ServerSecureChannel])
4. The Computer Name (The Domain Controller ex: DC01)
5. The Client Credential String (this will be 8 hexets of \x00 [16 Bytes of Zero])
6. Negotiation Flags (The following value observed from a Win10 client with Sign/Seal flags disabled: 0x212fffff Provided by Secura)

Step 4. The server will receive the NetrServerAuthenticate request and will compute the same request itself using it's known, good values. If the results are good, the server will send the required info back to the client. *[Figure 1. Step 4.]*

At this point the attempt to exploit the Zero Logon vulnerability is under way. The above steps above will be looped through a certain number of times to attempt to exploit the Zero Logon vulnerability. The actual exploit occurs at Step 3 and 4, this where we're hoping for the Server to a have the same computations as the client. This is where are 1-in-256 chance comes in.

Step 5. If the server calculates the same value, the client will re-verify and once mutual agreement is confirmed, they will agree on a session key. The session key will be used to encrypt communications between the client and the server, which means authentication is successful. *[Figure 1. Step 5]*

From there, normal RPC communications can occur.